01. Application startup

React frontend entry point and FastAPI service initialization.

Actual source excerpts for your report; not a standalone runnable project.
Long lines wrap visually. Use the Markdown companion for exact, copyable code.

frontend/src/main.jsx | lines 1-18

```
import { createRoot } from "react-dom/client";
import { HashRouter } from "react-router-dom";
import App from "./App";
import { AuthProvider } from "./auth";
import { registerGuardian } from "./lib/notify";
import { armAudioUnlock } from "./lib/sound";
import "./styles/global.css";

registerGuardian();
armAudioUnlock();

createRoot(document.getElementById("root")).render(
  <HashRouter>
    <AuthProvider>
      <App />
    </AuthProvider>
  </HashRouter>
);
```

backend/app/main.py | lines 1-46

```
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from .database import Base, SessionLocal, engine
from .ml.fatigue_model import load_model
from .routers import auth, dashboard, fatigue, interventions
from .seed import seed_demo


@asynccontextmanager
async def lifespan(_app: FastAPI):
    Base.metadata.create_all(bind=engine)
    load_model()
    db = SessionLocal()
    try:
        seed_demo(db)
    finally:
        db.close()
    yield


app = FastAPI(
    title="AURA — Fatigue Detection & Wellness Care",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router)
app.include_router(fatigue.router)
app.include_router(dashboard.router)
app.include_router(interventions.router)


@app.get("/api/health")
def health():
    return {"ok": True, "service": "aura"}
```

02. Authentication and access control

Password hashing, token creation and authenticated-user lookup.

backend/app/auth.py | lines 1-58

```
import hashlib
import hmac
import os
from datetime import datetime, timedelta

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from .config import ACCESS_TOKEN_EXPIRE_MINUTES, ALGORITHM, SECRET_KEY
from .database import get_db
from .models import User

bearer = HTTPBearer(auto_error=False)


def hash_password(password: str) -> str:
    salt = os.urandom(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 120_000)
    return salt.hex() + ":" + digest.hex()


def verify_password(password: str, stored: str) -> bool:
    try:
        salt_hex, digest_hex = stored.split(":")
        salt = bytes.fromhex(salt_hex)
        expected = bytes.fromhex(digest_hex)
        actual = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 120_000)
        return hmac.compare_digest(expected, actual)
    except Exception:
        return False


def create_token(user_id: int) -> str:
    payload = {
        "sub": str(user_id),
        "exp": datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
        "iat": datetime.utcnow(),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)


def get_current_user(
    creds: HTTPAuthorizationCredentials | None = Depends(bearer),
    db: Session = Depends(get_db),
) -> User:
    if creds is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Not signed in")
    try:
        data = jwt.decode(creds.credentials, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = int(data["sub"])
    except Exception as exc:
        raise HTTPException(status_code=401, detail="Invalid token") from exc
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(status_code=401, detail="User not found")
    return user
```

03. Database and session persistence

SQLite connection, database sessions and ORM user/work-session models.

backend/app/database.py | lines 1-35

```
from datetime import datetime

from sqlalchemy import create_engine, event
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from .config import DB_PATH

engine = create_engine(
    f"sqlite:///{DB_PATH}",
    connect_args={"check_same_thread": False},
)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)


@event.listens_for(engine, "connect")
def _sqlite_pragma(dbapi_connection, _connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()


class Base(DeclarativeBase):
    pass


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def utcnow() -> datetime:
    return datetime.utcnow()
```

backend/app/models.py | lines 9-40

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(80))
    email: Mapped[str] = mapped_column(String(160), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)

    work_start_hour: Mapped[int] = mapped_column(Integer, default=9)
    work_end_hour: Mapped[int] = mapped_column(Integer, default=18)
    break_sensitivity: Mapped[str] = mapped_column(String(20), default="balanced")
    enable_camera: Mapped[bool] = mapped_column(Boolean, default=True)
    enable_keyboard: Mapped[bool] = mapped_column(Boolean, default=True)
    enable_mouse: Mapped[bool] = mapped_column(Boolean, default=True)

    sessions: Mapped[list["WorkSession"]] = relationship(back_populates="user")


class WorkSession(Base):
    __tablename__ = "sessions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    started_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
    ended_at: Mapped[datetime | None] = mapped_column(DateTime, nullable=True)
    screen_minutes: Mapped[float] = mapped_column(Float, default=0)
    avg_fatigue: Mapped[float] = mapped_column(Float, default=0)
    peak_fatigue: Mapped[float] = mapped_column(Float, default=0)
    blink_total: Mapped[int] = mapped_column(Integer, default=0)
    break_count: Mapped[int] = mapped_column(Integer, default=0)
    skipped_breaks: Mapped[int] = mapped_column(Integer, default=0)
```

04. Feature schema and API communication

Validated multimodal inputs, browser API calls and prediction endpoint entry.

backend/app/schemas.py | lines 47-70

```
class FeatureIn(BaseModel):
    blink_rate: float = 16.0
    perclos: float = 0.08
    avg_ear: float = 0.28
    ear_std: float = 0.03
    head_pitch: float = -4.0
    head_yaw: float = 0.0
    head_roll: float = 0.0
    mar: float = 0.05
    yawn_count: float = 0
    typing_rpm: float = 120
    mouse_intensity: float = 0.4
    idle_ratio: float = 0.1
    continuous_screen_min: float = 15
    hour_of_day: float = 11
    session_min: float = 15
    posture_slouch: float = 0.15
    look_away_ratio: float = 0.05
    session_id: int | None = None
    personal_activity: bool = False
    behavioral_ready: bool = False
    behavioral_score: float = Field(default=0, ge=0, le=100, allow_inf_nan=False)
    typing_slowdown: float = Field(default=0, ge=0, le=1, allow_inf_nan=False)
    mouse_slowdown: float = Field(default=0, ge=0, le=1, allow_inf_nan=False)
```

frontend/src/api.js | lines 16-36

```
  if (auth && getToken()) headers.Authorization = `Bearer ${getToken()}`;
  const res = await fetch(path, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) {
    const message = data.detail || data.message || `Request failed (${res.status})`;
    throw new Error(typeof message === "string" ? message : JSON.stringify(message));
  }
  return data;
}

export async function api(path, opts = {}) {
  try {
    return await remoteApi(path, opts);
  } catch {
    return localApi(path, opts);
  }
}
```

backend/app/routers/fatigue.py | lines 48-58

```
@router.post("/fatigue/predict", response_model=PredictOut)
def predict(body: FeatureIn, user: User = Depends(get_current_user), db: Session = Depends(get_db)):
    payload = body.model_dump()
    result = predict_fatigue(payload)

    session_id = body.session_id
    if session_id:
        session = db.get(WorkSession, session_id)
        if not session or session.user_id != user.id:
            session_id = None
```

05. Face model loading and health checks

Local model/WASM loading, CPU inference, cancellation and freshness checks.

frontend/src/lib/faceEngine.js | lines 1-62

```
/** Assets are served with AURA, not fetched from Google/CDNs at camera start. */
export function visionAssetRoot(base = "./", page = document.baseURI) {
  return new URL(`${base}vision/`, page).href;
}

/** Dispose engines that finish after retry/unmount/timeout instead of publishing them. */
export function ownedTask(promise, { signal, timeoutMs = 30000, label = "Face engine", dispose = () => {} }
 = {}) {
  return new Promise((resolve, reject) => {
    let settled = false;
    let timer;
    const finish = (fn, value) => {
      settled = true; clearTimeout(timer); signal?.removeEventListener("abort", abort); fn(value);
    };
    const abort = () => { if (!settled) finish(reject, new Error(`${label} cancelled`)); };
    if (signal?.aborted) abort();
    else {
      signal?.addEventListener("abort", abort, { once: true });
      timer = setTimeout(() => { if (!settled) finish(reject, new Error(`${label} timed out. Click Retry
face detection.`)); }, timeoutMs);
    }
    Promise.resolve(promise).then(value => {
      if (settled) { try { dispose(value); } catch { /* already disposed */ } }
      else finish(resolve, value);
    }, error => { if (!settled) finish(reject, error); });
  });
}

export async function loadFaceEngine({ signal, base = "./", page, importVision = () =>
import("@mediapipe/tasks-vision"), fetcher = fetch } = {}) {
  const root = visionAssetRoot(base, page);
  const { FaceLandmarker, FilesetResolver } = await ownedTask(importVision(), { signal, label: "Face
library" });
  const controller = new AbortController();
  const abort = () => controller.abort();
  signal?.addEventListener("abort", abort, { once: true });
  const timer = setTimeout(abort, 30000);
  let bytes;
  try {
    if (signal?.aborted) throw new Error("Face engine cancelled");
    const response = await fetcher(`${root}face_landmarker.task`, { signal: controller.signal });
    if (!response.ok) throw new Error(`Face model download failed (${response.status}). Refresh the latest
AURA link and retry.`);
    bytes = new Uint8Array(await response.arrayBuffer());
    if (bytes.byteLength < 1000000) throw new Error("Face model is missing or incomplete. Refresh the
latest AURA link.");
  } catch (error) {
    if (controller.signal.aborted) throw new Error("Face model download interrupted or timed out. Click
Retry face detection.");
    throw error;
  } finally {
    clearTimeout(timer); signal?.removeEventListener("abort", abort);
  }
  const fileset = await ownedTask(FilesetResolver.forVisionTasks(root.replace(/\/$/, "")), { signal, label:
 "Face WASM loader" });
  // CPU avoids webcam/GPU delegate incompatibilities on common integrated GPUs.
  return ownedTask(FaceLandmarker.createFromOptions(fileset, {
    runningMode: "VIDEO", numFaces: 1,
    outputFaceBlendshapes: true, outputFacialTransformationMatrixes: true,
    baseOptions: { modelAssetBuffer: bytes, delegate: "CPU" },
  }), { signal, label: "Face tracking startup", dispose: engine => engine.close() });
}

export function trackingFresh(live, now = performance.now()) {
  return live?.trackingState === "healthy" && Number.isFinite(live.lastProcessedAt)
    && now >= live.lastProcessedAt && now - live.lastProcessedAt <= 5000;
}
export function faceUsable(live, now = performance.now()) {
  return trackingFresh(live, now) && live.present === true;
}
```

06. Camera startup and tracking watchdog

Owned engine startup, camera acquisition and stale-tracking cleanup.

frontend/src/hooks/useFaceMesh.js | lines 195-229

```
    async function startEngine() {
      const next = await loadFaceEngine({ signal: controller.signal, base: import.meta.env.BASE_URL });
      if (cancelled) { next.close(); return; }
      engine = next; landmarker.current = next;
      engineStarted = performance.now(); lastSuccess = 0; lastVideoTime = -1;
      setError(""); setStatus("tracking-loading");
    }
    async function boot() {
      setCamOn(false); setStatus("loading"); setError(""); clearTracking("starting");
      try {
        const video = await waitForVideoEl(videoRef, () => cancelled);
        if (cancelled) return;
        if (!video) throw new Error("Video element is not ready. Click Retry face detection.");
        let stream = video.srcObject;
        if (!liveTracks(stream)) {
          stream = await openCamera();
          if (cancelled) { stream.getTracks().forEach(t => t.stop()); return; }
          video.srcObject = stream;
        }
        ownedStream = stream;
        await playVideo(video);
        if (cancelled) return;
        setCamOn(true); setStatus("tracking-loading");
        await startEngine();
        if (cancelled) return;
        blinker.current = new BlinkTracker(); actor.current = new FaceActionTracker();
        frames.current = 0; fpsAt.current = performance.now();
        loop();
      } catch (err) {
        if (cancelled) return;
        failed = true; closeEngine();
        console.warn("Camera / face tracking unavailable", err);
        const hasCam = liveTracks(videoRef.current?.srcObject);
        setCamOn(hasCam); setError(friendlyCamError(err));
        setStatus(hasCam ? "tracking-error" : "idle"); clearTracking("error");
```

frontend/src/hooks/useFaceMesh.js | lines 349-364

```
    const watchdog = setInterval(() => {
      if (!cancelled && !failed && engine && !recovering && performance.now() - (lastSuccess ||
engineStarted) > 5000) {
        setStatus("tracking-paused"); clearTracking("stale");
        setError("Camera frames or face processing paused. Return to this tab or click Retry face
detection. Away timer and fatigue scoring are paused.");
      }
    }, 1000);
    boot();
    return () => {
      cancelled = true; controller.abort();
      clearTimeout(timer.current); clearInterval(watchdog); closeEngine();
      ownedStream?.getTracks().forEach(t => t.stop());
      if (videoRef.current?.srcObject === ownedStream) videoRef.current.srcObject = null;
    };
    // Each retry owns its engine/stream; late startup cannot overwrite a newer retry.
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [enabled, retryToken]);
```

07. Real-time facial feature extraction

Fresh video-frame inference, eye/mouth features, pose and action tracking.

frontend/src/hooks/useFaceMesh.js | lines 249-288

```
      try {
        const video = videoRef.current, canvas = canvasRef.current;
        if (!engine || recovering || !video || video.readyState < 2 || !video.videoWidth ||
video.currentTime === lastVideoTime) return;
        const now = performance.now();
        const result = engine.detectForVideo(video, now);
        lastVideoTime = video.currentTime; lastSuccess = now; processed++;
        setStatus("live"); setError("");
        frames.current++;
        let fps = 0;
        if (now - fpsAt.current >= 1000) {
          fps = Math.round(frames.current * 1000 / (now - fpsAt.current));
          frames.current = 0; fpsAt.current = now;
        }
        const face = result.faceLandmarks?.[0];
        const ctx = canvas?.getContext("2d");
        if (canvas && (canvas.width !== video.videoWidth || canvas.height !== video.videoHeight)) {
          canvas.width = video.videoWidth; canvas.height = video.videoHeight;
        }
        if (face) {
          blinker.current.setGlasses(glassesRef.current);
          const ear = bothEars(face);
          const mar = mouthAspect(face);
          const blends = blendMap(result);
          const bscore = blendScore(blends);
          const matrix = result.facialTransformationMatrixes?.[0]?.data;
          const pose = matrix ? poseFromMatrix(matrix) : poseFromLandmarks(face);
          const blink = blinker.current.push(ear.avg, now, bscore);
          const glasses = glassesRef.current || blink.glassesLikely;
          const leftClosed = eyeIsClosed(ear.left, blink.openBaseline, blends.eyeBlinkLeft || 0, glasses);
          const rightClosed = eyeIsClosed(ear.right, blink.openBaseline, blends.eyeBlinkRight || 0,
glasses);
          const actions = actor.current.push({
            closed: blink.closed,
            blinked: blink.blinked,
            mar,
            pitch: pose.pitch,
            yaw: pose.yaw,
            blends,
            speaking: speakingRef.current,
            now,
          });
```

frontend/src/hooks/useFaceMesh.js | lines 297-322

```
          setLive((s) => ({
            trackingState: "healthy",
            lastProcessedAt: now,
            processedFrames: processed,
            ear: ear.avg,
            earLeft: ear.left,
            earRight: ear.right,
            leftClosed,
            rightClosed,
            blinkRate: blink.blinkRate,
            perclos: blink.perclos,
            earStd: blink.earStd,
            pitch: pose.pitch,
            yaw: pose.yaw,
            roll: pose.roll,
            mar,
            yawns: actions.yawn.recentCount,
            jawOpen: blends.jawOpen || 0,
            yawnActive: actions.yawn.active,
            yawnStatus: actions.yawn.status,
            scoringMar: actions.yawn.scoringMar,
            yawnPending: actions.yawn.pending,
            mouthOpenMs: actions.yawn.openMs,
            slouch: slouchFromPose(pose.pitch, pose.roll),
            present: true,
            blinks: blink.blinks,
```

08. Eye and mouth landmark geometry

EAR, per-eye closure, corrected lip pairs, MAR and matrix-based head pose.

frontend/src/lib/vision.js | lines 1-59

```
import { YawnDetector } from "./yawn.js";

/** MediaPipe Face Mesh indices — aligned with backend/app/ml/ear.py */
export const LEFT_EYE = [33, 160, 158, 133, 153, 144];
export const RIGHT_EYE = [362, 385, 387, 263, 373, 380];
/** Denser lids — more stable when spectacle frames hide a corner. */
export const LEFT_EYE_DENSE = [33, 246, 161, 160, 159, 158, 157, 173, 133, 155, 154, 153, 145, 144,
163, 7];
export const RIGHT_EYE_DENSE = [362, 398, 384, 385, 386, 387, 388, 466, 263, 249, 390, 373, 374, 380,
381, 382];
// aspectRatio pairs p2↔p6 and p3↔p5: use the matching lower lip on each side.
// Swapping 178/402 measures diagonals and keeps closed-mouth MAR falsely high.
export const MOUTH = [61, 81, 311, 291, 402, 178];
export const FACE_OVAL = [10, 338, 297, 332, 284, 251, 389, 356, 454, 323, 361, 288, 397, 365, 379,
378, 400, 377, 152, 148, 176, 149, 150, 136, 172, 58, 132, 93, 234, 127, 162, 21, 54, 103, 67, 109];

export const EAR_THRESH = 0.21;
export const MAR_THRESH = 0.55;

export function dist(a, b) {
  const dx = a.x - b.x;
  const dy = a.y - b.y;
  const dz = (a.z || 0) - (b.z || 0);
  return Math.hypot(dx, dy, dz);
}

export function aspectRatio(landmarks, idx) {
  const [p1, p2, p3, p4, p5, p6] = idx.map((i) => landmarks[i]);
  return (dist(p2, p6) + dist(p3, p5)) / (2 * (dist(p1, p4) + 1e-6));
}

export function bothEars(landmarks) {
  const left = aspectRatio(landmarks, LEFT_EYE);
  const right = aspectRatio(landmarks, RIGHT_EYE);
  return { left, right, avg: (left + right) / 2 };
}

export function eyeIsClosed(earValue, openBaseline = 0.28, blinkScore = 0, glasses = false) {
  const drop = glasses ? 0.78 : 0.72;
  const blendThresh = glasses ? 0.38 : 0.5;
  const byEar = earValue < Math.max(0.12, openBaseline * drop);
  const byBlend = blinkScore >= blendThresh;
  return glasses ? byBlend || byEar : byEar || (byBlend && earValue < openBaseline * 0.85);
}

export function mouthAspect(landmarks) {
  return aspectRatio(landmarks, MOUTH);
}

/** Euler angles (degrees) from MediaPipe 4x4 facial transformation matrix. */
export function poseFromMatrix(matrix) {
  if (!matrix || matrix.length < 16) return { pitch: 0, yaw: 0, roll: 0 };
  const r00 = matrix[0];
  const r10 = matrix[1];
  const r20 = matrix[2];
  const r21 = matrix[6];
  const r22 = matrix[10];
  const pitch = Math.atan2(-r21, r22) * (180 / Math.PI);
  const yaw = Math.atan2(r20, Math.hypot(r00, r10)) * (180 / Math.PI);
  const roll = Math.atan2(r10, r00) * (180 / Math.PI);
  return { pitch, yaw, roll };
}
```

09. Conservative yawn filtering

Mouth/jaw agreement, speech-like movement suppression and closure confirmation.

frontend/src/lib/yawn.js | lines 62-101

```
    const hasJaw = Number.isFinite(jaw);
    // Require supporting evidence, but do not require two simultaneously extreme scores.
    // Jaw-led and geometry-led routes accommodate faces with different signal ranges.
    const wide = hasJaw
      ? (mar >= 0.45 && jaw >= 0.35) || (jaw >= 0.70 && mar >= 0.30) || (mar >= 0.72 && jaw >= 0.25)
      : mar >= 0.65;
    const released = mar < 0.40 && (!hasJaw || jaw < 0.35);
    if (this.previousWide !== null && wide !== this.previousWide) this.transitions.push(now);
    this.previousWide = wide;
    this.transitions = this.transitions.filter(t => now - t <= 2500);
    if (this.transitions.length >= 5) this.speechUntil = now + 1500;
    if (now < this.speechUntil) {
      this.startedAt = this.closedAt = this.neutralSince = null;
      this.armed = this.ready = false;
      this.samples = [];
      return result('speech-like');
    }

    if (released) {
      if (this.neutralSince === null) this.neutralSince = now;
      if (this.startedAt !== null) {
        if (this.closedAt === null) this.closedAt = now;
        if (now - this.closedAt >= 400) {
          const accepted = this.ready && now - this.lastCountAt >= 6000;
          if (accepted) {
            this.lastAcceptedMar = this.peakMar;
            this.count += 1;
            this.times.push(now);
            this.lastCountAt = now;
          }
          this.startedAt = this.closedAt = null;
          this.ready = this.armed = false;
          this.samples = [];
          return result(accepted ? 'counted' : 'monitoring', accepted);
        }
      } else if (now - this.neutralSince >= 400) {
        this.armed = true;
      }
      return result(this.startedAt !== null ? 'closing' : this.armed ? 'monitoring' : 'settling');
    }
```

frontend/src/lib/yawn.js | lines 120-135

```
    this.peakMar = Math.max(this.peakMar, mar);
    this.samples.push({ t: now, mar, jaw: hasJaw ? jaw : null, wide });
    this.samples = this.samples.filter(s => now - s.t <= YAWN_HOLD_MS);
    const mean = key => this.samples.reduce((sum, s) => sum + (s[key] ?? 0), 0) / this.samples.length;
    const std = key => {
      const avg = mean(key);
      return Math.sqrt(this.samples.reduce((sum, s) => sum + ((s[key] ?? 0) - avg) ** 2, 0) /
this.samples.length);
    };
    const wideRatio = this.samples.filter(s => s.wide).length / this.samples.length;
    // Latch qualification: a natural, gradual closing phase must not erase a valid hold.
    // Speech suppression, face loss and excessive duration still cancel the whole cycle.
    this.ready ||= now - this.startedAt >= YAWN_HOLD_MS && wide && wideRatio >= 0.80
      && std('mar') <= 0.14 && (!hasJaw || std('jaw') <= 0.16);
    return result(this.ready ? 'candidate' : 'checking');
  }
}
```

10. Typing and mouse measurement

Single-keydown counting, pointer distance/time and focused input collection.

extension/activity-sampler.js | lines 1-38

```
/* Numeric activity only: never transmit text, key identities, or cursor positions.
   Only the last pointer point is held transiently to calculate distance. */
(function (root) {
  function isTypingKey(e) {
    return !e.repeat && !e.ctrlKey && !e.metaKey && !e.altKey && e.target?.type !== "password"
      && (e.key?.length === 1 || ["Backspace", "Delete", "Enter"].includes(e.key));
  }
  class InputSampler {
    constructor(now = Date.now()) { this.reset(now); }
    reset(now) {
      this.at = now; this.keys = 0; this.distance = 0; this.movingMs = 0; this.point = null;
    }
    key(e) { if (isTypingKey(e)) this.keys++; }
    move(e, now = Date.now()) {
      const x = Number(e.clientX), y = Number(e.clientY);
      if (!Number.isFinite(x) || !Number.isFinite(y)) return;
      if (this.point) {
        const dt = now - this.point.at;
        const distance = Math.hypot(x - this.point.x, y - this.point.y);
        if (dt > 0 && dt <= 250 && distance > 0 && distance < 4000) {
          this.distance += distance; this.movingMs += dt;
        }
      }
      this.point = { x, y, at: now };
    }
    sample(now = Date.now()) {
      const dt = now - this.at;
      const result = dt > 0 && dt <= 2500 ? {
        at: now, dt, keys: Math.min(100, this.keys),
        mouseDistance: Math.min(100000, this.distance), mouseMs: Math.min(dt, this.movingMs),
      } : null;
      this.at = now; this.keys = 0; this.distance = 0; this.movingMs = 0;
      if (this.point && now - this.point.at > 250) this.point = null;
      return result;
    }
  }
  root.AuraActivitySampler = Object.freeze({ InputSampler, isTypingKey });
})(globalThis);
```

frontend/src/hooks/useActivity.js | lines 20-36

```
  useEffect(() => {
    tellGuardian({ type: "AURA_ACTIVITY_PREFS", enabled, keyboard, mouse });
    if (!enabled) { setStats(tracker.current.read()); return undefined; }
    const sampler = new InputSampler();
    let lastSync = Date.now();
    const focused = () => !document.hidden && document.hasFocus();
    const onKey = e => { if (keyboard && focused()) sampler.key(e); };
    const onMove = e => { if (mouse && focused()) sampler.move(e); };
    const onFocus = () => {
      sampler.reset(Date.now());
      tellGuardian({ type: "AURA_ACTIVITY_PREFS", enabled, keyboard, mouse });
    };
    const onMessage = e => {
      // Local and Guardian samples are alternatives, never summed/maxed together.
      if (e.source !== window) return;
      if (e.data?.type === "AURA_ACTIVITY_READY") {
        tellGuardian({ type: "AURA_ACTIVITY_PREFS", enabled, keyboard, mouse }); return;
```

11. Personal starting-pace baseline

First active-minute reference, sustained slowdown and bounded score adjustment.

frontend/src/lib/activity.js | lines 1-47

```
import "../../../extension/activity-sampler.js";
export const { InputSampler, isTypingKey } = globalThis.AuraActivitySampler;
const clamp = (v, lo = 0, hi = 1) => Math.min(hi, Math.max(lo, v));
export const CALIBRATION_MS = 60000;
export const SLOW_HOLD_MS = 30000;
const channel = () => ({ ms: 0, amount: 0, denominator: 0, baseline: null, slowSince: null });

/** Per-monitor-session reference, frozen after 60 active seconds per input channel. */
export class PersonalPace {
  constructor(now = Date.now()) { this.reset(now); }
  reset(now = Date.now()) {
    this.typing = channel(); this.mouse = channel(); this.suspend(now);
  }
  suspend(now = Date.now()) {
    this.rows = []; this.started = now; this.lastRead = now;
    this.typing.slowSince = this.mouse.slowSince = null;
    this.lastSample = -Infinity;
  }
  ingest(sample, now = Date.now(), { keyboard = true, mouse = true } = {}) {
    if (!sample || ![sample.at, sample.dt, sample.keys, sample.mouseDistance,
sample.mouseMs].every(Number.isFinite)) return false;
    if (sample.at > now + 100 || now - sample.at > 4000 || sample.at <= this.lastSample || sample.dt <= 0 ||
sample.dt > 2500) return false;
    this.lastSample = sample.at;
    const row = { at: sample.at, dt: sample.dt, keys: keyboard ? clamp(sample.keys, 0, 100) : 0,
      distance: mouse ? clamp(sample.mouseDistance, 0, 100000) : 0,
      moving: mouse ? clamp(sample.mouseMs, 0, sample.dt) : 0 };
    this.rows.push(row);
    this.learn(this.typing, row.keys, row.dt, row.keys > 0 ? row.dt : 0);
    this.learn(this.mouse, row.distance, row.moving, row.distance > 0 && row.moving > 0 ? row.dt : 0);
    return true;
  }
  learn(c, amount, denominator, activeMs) {
    if (c.baseline !== null || !activeMs || !denominator) return;
    // Freeze the first active minute; do not let later fatigue lower the reference.
    const fraction = Math.min(1, (CALIBRATION_MS - c.ms) / activeMs);
    c.amount += amount * fraction; c.denominator += denominator * fraction; c.ms += activeMs * fraction;
    if (c.ms >= CALIBRATION_MS) c.baseline = c.amount / c.denominator * 1000;
  }
  compare(c, amount, denominator, activeMs, recent, now) {
    const current = denominator > 0 ? amount / denominator * 1000 : 0;
    const active = c.baseline !== null && activeMs >= 20000 && recent;
    const drop = active ? clamp(1 - current / c.baseline) : 0;
    if (!active || drop <= .15) c.slowSince = null;
    else if (c.slowSince === null) c.slowSince = now;
    const held = c.slowSince === null ? 0 : now - c.slowSince;
    return { baseline: c.baseline, current, progress: clamp(c.ms / CALIBRATION_MS), active, drop,
      heldSeconds: Math.floor(held / 1000), evidence: active && held >= SLOW_HOLD_MS ? clamp((drop - .15) / .60) :
0 };
  }
```

frontend/src/lib/activity.js | lines 54-64

```
    const typing = this.compare(this.typing, sum("keys"), sum("dt", r => r.keys > 0), sum("dt", r => r.keys > 0),
recent(r => r.keys > 0), now);
    const mouse = this.compare(this.mouse, sum("distance"), sum("moving"), sum("dt", r => r.distance > 0 &&
r.moving > 0), recent(r => r.distance > 0), now);
    const eligible = [typing, mouse].filter(c => c.active);
    const behaviorScore = eligible.length ? 100 * eligible.reduce((n, c) => n + c.evidence, 0) / eligible.length :
0;
    const span = Math.max(1000, Math.min(60000, now - this.started));
    const typingRpm = Math.round(sum("keys") * 60000 / span);
    const observed = sum("dt"), idle = sum("dt", r => r.keys === 0 && r.distance === 0);
    return { typingRpm, typingWpm: Math.round(typingRpm / 5), mouseSpeed: mouse.current,
      mouseIntensity: clamp(mouse.current / 1000), idleRatio: observed ? clamp(idle / observed) : 0,
      lookAwayRatio: 0, typing, mouse, behaviorScore: +behaviorScore.toFixed(1), behaviorReady: eligible.length > 0
 };
  }
```

frontend/src/lib/activity.js | lines 71-75

```
export function activityAdjustment(f) {
  if (f.personal_activity !== true || f.behavioral_ready !== true) return 0;
  const score = Number(f.behavioral_score);
  return Number.isFinite(score) ? +((clamp(score, 0, 100) / 100) * 20).toFixed(1) : 0;
}
```

12. LightGBM training and fatigue scoring

Synthetic-data training, prediction and the explicit personal-pace adjustment.

backend/app/ml/train.py | lines 108-142

```
def train(n: int = 14000) -> dict:
    ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)
    X, y = synthesize(n)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.18, random_state=7)

    model = lgb.LGBMRegressor(
        n_estimators=420,
        learning_rate=0.045,
        num_leaves=48,
        max_depth=8,
        min_child_samples=24,
        subsample=0.85,
        colsample_bytree=0.9,
        reg_lambda=1.2,
        random_state=7,
        verbosity=-1,
    )
    model.fit(X_train, y_train)

    pred = model.predict(X_test)
    metrics = {
        "mae": float(mean_absolute_error(y_test, pred)),
        "r2": float(r2_score(y_test, pred)),
        "n_train": int(len(X_train)),
        "n_test": int(len(X_test)),
        "features": FEATURE_COLUMNS,
        "importances": {
            name: float(val)
            for name, val in zip(FEATURE_COLUMNS, model.feature_importances_)
        },
    }

    joblib.dump(model, ARTIFACTS_DIR / "fatigue_lgbm.pkl")
    (ARTIFACTS_DIR / "metrics.json").write_text(json.dumps(metrics, indent=2))
    return metrics
```

backend/app/ml/fatigue_model.py | lines 160-178

```
def predict_fatigue(payload: dict) -> dict:
    model = get_model()
    X = features_to_array(model_payload(payload))
    raw = float(model.predict(X)[0])
    adjustment = activity_adjustment(payload)
    score = float(np.clip(raw + adjustment, 0, 100))
    level = score_to_level(score)
    # Confidence shrinks when inputs are extreme / sparse
    ear = float(payload.get("avg_ear") or 0)
    presence = 1.0 if 0.12 < ear < 0.4 else 0.55
    confidence = float(np.clip(0.62 + 0.28 * presence, 0.45, 0.94))
    return {
        "fatigue_score": round(score, 1),
        "level": level,
        "confidence": round(confidence, 2),
        "drivers": _drivers(payload, score),
        "metrics": _metrics,
        "activity_adjustment": adjustment,
    }
```

backend/app/ml/activity.py | lines 1-16

```
"""Transparent personal-baseline adjustment, not a newly trained ML feature."""
import math


def activity_adjustment(payload: dict) -> float:
    if payload.get("personal_activity") is not True or payload.get("behavioral_ready") is not True:
        return 0.0
    try:
        value = float(payload.get("behavioral_score", 0))
    except (ValueError, TypeError):
        return 0.0
    return round(min(100.0, max(0.0, value)) * 0.2, 1) if math.isfinite(value) else 0.0


def model_payload(payload: dict) -> dict:
    if payload.get("personal_activity") is not True:
```

13. LinUCB intervention personalization

Upper-confidence selection, reward updates and break-trigger gating.

backend/app/ml/bandit.py | lines 89-103

```
def _ucb(state_arm: dict, x: np.ndarray) -> float:
    A = np.array(state_arm["A"], dtype=np.float64)
    b = np.array(state_arm["b"], dtype=np.float64)
    try:
        A_inv = np.linalg.inv(A)
    except np.linalg.LinAlgError:
        A_inv = np.linalg.pinv(A)
    theta = A_inv @ b
    mean = float(theta @ x)
    bonus = ALPHA * math.sqrt(float(x @ A_inv @ x) + 1e-8)
    return mean + bonus


def _heuristic_rank(arm: str, features: dict, score: float) -> float:
    """Cold-start prior so the first sessions are not random."""
```

backend/app/ml/bandit.py | lines 123-163

```
def choose_arm(db: Session, user_id: int, features: dict, score: float) -> tuple[str, float]:
    state = load_state(db, user_id)
    x = build_context(features, score)
    scored = []
    for arm in ARM_IDS:
        ucb = _ucb(state[arm], x)
        prior = _heuristic_rank(arm, features, score)
        n = state[arm].get("n", 0)
        # Blend prior until the arm has been tried a few times
        mix = 1 / (1 + n / 3)
        scored.append((arm, (1 - mix) * ucb + mix * prior))
    scored.sort(key=lambda t: t[1], reverse=True)
    return scored[0]


def update(db: Session, user_id: int, arm: str, features: dict, score: float, reward: float) -> None:
    if arm not in ARM_IDS:
        return
    state = load_state(db, user_id)
    x = build_context(features, score)
    A = np.array(state[arm]["A"], dtype=np.float64)
    b = np.array(state[arm]["b"], dtype=np.float64)
    A += np.outer(x, x)
    b += reward * x
    state[arm]["A"] = A.tolist()
    state[arm]["b"] = b.tolist()
    state[arm]["n"] = int(state[arm].get("n", 0)) + 1
    save_state(db, user_id, state)


def should_interrupt(score: float, sensitivity: str, minutes_since_break: float, features: dict) ->
bool:
    gates = {"gentle": 68, "balanced": 52, "assertive": 40}
    threshold = gates.get(sensitivity, 52)
    if minutes_since_break < 8:
        return False
    if score >= 78:
        return True
    screen = float(features.get("continuous_screen_min") or 0)
    if screen >= 55 and score >= threshold - 8:
        return True
    return score >= threshold
```

backend/app/ml/bandit.py | lines 192-197

```
def reward_from_feedback(status: str, fatigue_before: float, fatigue_after: float | None) -> float:
    base = {"completed": 1.0, "partial": 0.35, "skipped": -0.25, "dismissed": -0.35}.get(status, 0.0)
    if fatigue_after is not None and status == "completed":
        delta = fatigue_before - fatigue_after
        base += float(np.clip(delta / 40.0, -0.2, 0.5))
    return float(np.clip(base, -0.5, 1.5))
```

14. Selectable beep sounds and preview

Saved sound/volume preferences, preview playback, fatigue mute and away tone.

frontend/src/lib/sound.js | lines 11-32

```
export function getAudioSettings() { return catalog.readPreferences(localStorage); }
export function isSoundOn() { return getAudioSettings().enabled; }
export function getVolume() { return getAudioSettings().volume; }
export function getBeepSound() { return getAudioSettings().soundId; }
export function isAudioReady() { return unlocked; }

export function syncAudioPreferences() {
  const settings = getAudioSettings();
  tellGuardian({ type: "AURA_AUDIO_SETTINGS", settings });
  try { window.dispatchEvent(new CustomEvent("aura-audio-settings", { detail: settings })); } catch {
/* not a browser */ }
}
export function setSoundOn(on) {
  localStorage.setItem("aura_sound_on", on ? "1" : "0");
  syncAudioPreferences();
}
export function setVolume(value) {
  localStorage.setItem("aura_sound_vol", String(catalog.sanitizePreferences({ volume: value
}).volume));
  syncAudioPreferences();
}
export function setBeepSound(id) {
  localStorage.setItem("aura_beep_sound", catalog.getSound(id).id);
  syncAudioPreferences();
```

frontend/src/lib/sound.js | lines 53-89

```
function playChoice(id, kind = "fatigue") {
  unlockAudio();
  const c = audioCtx();
  if (c?.state === "running") return catalog.play(c, id, getVolume(), kind);
  // One audio path at a time: no parallel WAV/oscillator amplification or per-tab stacking.
  const audio = new Audio(wavUrl(id, kind));
  let cancelled = false;
  audio.volume = getVolume();
  audio.play().catch(() => {
    if (cancelled) return;
    try { window.dispatchEvent(new Event("aura-audio-blocked")); } catch { /* ignore */ }
  });
  return () => { cancelled = true; audio.pause(); audio.currentTime = 0; };
}
export function stopBeepPreview() {
  stopPreview?.();
  stopPreview = null;
}
export function previewBeep(id = getBeepSound()) {
  stopBeepPreview();
  try {
    stopPreview = playChoice(catalog.getSound(id).id);
    return true;
  } catch { return false; }
}
export function playFatigueBeep(_urgent = false) {
  if (!isSoundOn()) return false;
  stopBeepPreview();
  try { playChoice(getBeepSound()); return true; } catch { return false; }
}
export function playRitualChime() {
  return playFatigueBeep();
}
/** Fixed, recognizable away alarm: F8 cannot mute it; Stop alarm halts its repeating timer. */
export function playAlarmBeep() {
  stopBeepPreview();
  try { playChoice(getBeepSound(), "away"); return true; } catch { return false; }
```

15. Fatigue alerts and reminder scheduling

Sustained score-75 warnings and persisted Guardian scheduled notifications.

frontend/src/pages/Monitor.jsx | lines 212-241

```
      const score = predRef.current?.fatigue_score ?? 0;
      const watching = faceUsable(liveRef.current) || statusRef.current === "lab";
      if (runningRef.current && watching && score >= 75) {
        if (!fatigueSince.current) fatigueSince.current = performance.now();
        const held = (performance.now() - fatigueSince.current) / 1000;
        setFatigueHold(held);
        document.title = `AURA · ${Math.round(score)} · ${Math.floor(held)}s fatigue`;
        const microsleep = (liveRef.current?.closedMs || 0) > 1000;
        if (
          (held >= 30 || microsleep) &&
          performance.now() - lastBeep.current > (microsleep ? 8000 : 30000) &&
          performance.now() > snoozeUntil.current
        ) {
          lastBeep.current = performance.now();
          const ok = playFatigueBeep(score >= 75);
          tellBeep(score >= 75 ? "fatigue" : "fatigue", false);
          setFlash(true);
          window.setTimeout(() => setFlash(false), 700);
          pushWindows(
            `AURA · fatigue for ${Math.floor(held)} seconds`,
            `Score ${Math.round(score)}. Look away, blink fully, or take a ritual. Press F8 to mute the
beep.`,
            "aura-hold"
          );
          if (!ok) setHint("Beep blocked — click Test beep + Windows once, then wait 30s of fatigue.");
        }
      } else {
        fatigueSince.current = null;
        setFatigueHold(0);
        if (!alarmOnRef.current) document.title = "AURA — Fatigue Detection";
      }
```

extension/background.js | lines 204-239

```
async function restoreSchedule() {
  const { emberItems = [] } = await chrome.storage.local.get("emberItems");
  const alarms = await chrome.alarms.getAll();
  for (const alarm of alarms) {
    if (alarm.name.startsWith("ember:")) await chrome.alarms.clear(alarm.name);
  }
  for (const item of emberItems) {
    if (item.done || item.notified) continue;
    const when = Date.parse(item.when);
    if (!Number.isFinite(when)) continue;
    if (when <= Date.now()) await deliverSchedule(item.id);
    else await chrome.alarms.create(`ember:${item.id}`, { when });
  }
}
async function deliverSchedule(id) {
  const { emberItems = [] } = await chrome.storage.local.get("emberItems");
  const item = emberItems.find(row => row.id === id);
  if (!item || item.done || item.notified) return;
  // Ignore stale alarms after editing a date.
  if (Date.parse(item.when) > Date.now()) {
    await chrome.alarms.create(`ember:${id}`, { when: Date.parse(item.when) });
    return;
  }
  await chrome.notifications.create(`ember:${id}`, {
    type: "basic", iconUrl: "icon128.png", title: "Ember Docket · scheduled reminder",
    message: `${item.title}${item.note ? " — " + item.note : ""}`,
    priority: 2, requireInteraction: true,
  });
  item.notified = true;
  await chrome.storage.local.set({ emberItems });
  broadcast({ type: "AURA_SCHEDULE_ACK", items: emberItems });
  broadcast({ type: "AURA_SCHEDULE_DUE", item });
  await enqueueAudio(() => playGuardianAudio("docket"));
}
chrome.runtime.onStartup.addListener(() => enqueueSchedule(restoreSchedule));
chrome.runtime.onInstalled.addListener(() => enqueueSchedule(restoreSchedule));
```